Math Nuggets for GEN AI

Bit Quantization for Large Language Models: Understanding Digital Precision and Memory Efficiency

By Dr. Anish Roychowdhury, Plaksha University

September 20, 2025

Introduction

Imagine trying to paint a masterpiece but being told you can only use 16 colors instead of the millions available in your palette. This is essentially what quantization does to Large Language Models (LLMs) - it forces them to represent complex numerical information using fewer "colors" or precision levels.

While this might seem limiting, it's a powerful technique that makes these massive AI systems practical to run on everyday computers. This note explores how we can dramatically reduce the memory requirements of LLMs while maintaining their intelligence.

1. Understanding Digital Number Systems

1.1 From Decimal to Binary: The Foundation

In our everyday life, we use the decimal system (base-10) with digits 0-9. Computers, however, operate in binary (base-2) using only 0s and 1s.

Decimal Example: The number 13 means $1 \times 10^1 + 3 \times 10^0 = 10 + 3 = 13$

Binary Example: The same number 13 in binary is 1101, meaning: $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13$

1.2 Floating-Point Representation

Real numbers (like 3.14159) are stored using floating-point representation. Think of it like scientific notation:

Scientific Notation: 6.022×10^{23} (Avogadro's number)

Computer Storage: Numbers are stored as: $\pm\text{mantissa} \times 2^{\text{exponent}}$

32-bit Float (FP32): Uses 32 bits total

- 1 bit for sign (+ or -)
- 8 bits for exponent (scale)
- 23 bits for mantissa (precision)

This gives us approximately 7 decimal digits of precision and can represent numbers from $\pm 1.4 \times 10^{-45}$ to $\pm 3.4 \times 10^{38}$.

1.3 The Memory Challenge

A modern LLM like GPT-3 has 175 billion parameters. If each parameter uses 32 bits (4 bytes):

$$\text{Memory Required} = 175 \times 10^9 \times 4 \text{ bytes} = 700 \text{ GB} \tag{1}$$

This is more memory than most computers have! This is where quantization becomes crucial.

2. What is Quantization?

2.1 The Intuitive Explanation

Quantization is like reducing the color depth of a digital image. A 24-bit image can show 16.7 million colors, while an 8-bit image shows only 256 colors. The image still looks recognizable, but uses much less storage.

Similarly, quantization reduces the precision of numbers in neural networks from high-precision (32-bit) to lower precision (8-bit, 4-bit, or even lower).

2.2 Why Does This Work?

Neural networks are surprisingly robust to precision reduction because:

- Many weights contribute to each decision (redundancy)
- Small errors in individual weights often cancel out
- The network learns patterns, not exact numerical relationships
- Most information is captured by the relative magnitudes of weights

3. Mathematical Framework of Quantization

3.1 Linear Quantization Explained

Imagine you have a thermometer that measures from -10°C to +50°C, but you can only make 16 marks on it (4 bits = $2^4 = 16$ levels).

For a continuous value x in range $[x_{min}, x_{max}]$, linear quantization works as:

Step 1: Find the scale

$$s = \frac{x_{max} - x_{min}}{2^b - 1} \tag{2}$$

Step 2: Quantize the value

$$x_q = \text{round}\left(\frac{x - x_{min}}{s}\right) \cdot s + x_{min} \tag{3}$$

Temperature Example:

- Range: -10°C to +50°C (60° total range)
- 4 bits = 16 levels
- Scale: $s = 60/15 = 4$ per level
- Temperature 23°C becomes: round((23-(-10))/4) × 4 + (-10) = 22°C
- Error: 1°C

3.2 Symmetric Quantization

For neural network weights that are roughly centered around zero, we use symmetric quantization:

$$s = \frac{2 \times |x_{max}|}{2^b - 1} \tag{4}$$

This ensures equal precision for positive and negative values.

4. Detailed Example: 32-bit to 4-bit Weight Quantization

4.1 The Original Weights

Consider a small section of a neural network with these 32-bit floating-point weights:

$$W_{32} = \begin{bmatrix} 0.8247 & -0.3891 & 0.6523 \\ -0.2145 & 0.9876 & -0.7234 \\ 0.4567 & -0.8901 & 0.1234 \end{bmatrix} \tag{5}$$

These represent learned patterns - perhaps how much attention to pay to different input features.

4.2 Quantization Process Step-by-Step

Step 1: Find the maximum absolute value

$x_{max} = \max(|W|) = 0.9876$

Step 2: Calculate scale for 4-bit quantization

With 4 bits, we have $2^4 = 16$ possible values: -7, -6, ..., 0, ..., 6, 7 (using one bit for sign)

$$s = \frac{2 \times 0.9876}{15} = 0.1317 \tag{6}$$

Step 3: Apply quantization

For each weight w:

- $w = 0.8247 \rightarrow \text{round}(0.8247/0.1317) \times 0.1317 = 6 \times 0.1317 = 0.7902$
- $w = -0.3891 \rightarrow \text{round}(-0.3891/0.1317) \times 0.1317 = -3 \times 0.1317 = -0.3951$

4.3 The Quantized Result

$$W_4 = \begin{bmatrix} 0.7902 & -0.3951 & 0.6585 \\ -0.1317 & 0.9902 & -0.7902 \\ 0.3951 & -0.9219 & 0.1317 \end{bmatrix} \tag{7}$$

4.4 Understanding the Error

The quantization error shows how much information we lost:

$$E = W_{32} - W_4 = \begin{bmatrix} 0.0345 & 0.0060 & -0.0062 \\ -0.0828 & -0.0026 & 0.0668 \\ 0.0616 & 0.0318 & -0.0083 \end{bmatrix} \tag{8}$$

Average error magnitude: $\frac{1}{9}\sum_{i,j}|E_{ij}| = 0.0350$ (about 3.5

5. Computational Impact: A Real Calculation

5.1 Forward Pass Comparison

Let's see how quantization affects actual computations. Consider processing an input vector through our weight matrix:

Input: $\mathbf{x} = [1.2, -0.8, 0.5]^T$ (representing processed features from text)

32-bit computation:

$$y_1 = 0.8247 \times 1.2 + (-0.3891) \times (-0.8) + 0.6523 \times 0.5 = 1.6389 \tag{9}$$

$$y_2 = -0.2145 \times 1.2 + 0.9876 \times (-0.8) + (-0.7234) \times 0.5 = -1.4730 \tag{10}$$

$$y_3 = 0.4567 \times 1.2 + (-0.8901) \times (-0.8) + 0.1234 \times 0.5 = 1.6097 \tag{11}$$

4-bit computation:

$$y_1 = 0.7902 \times 1.2 + (-0.3951) \times (-0.8) + 0.6585 \times 0.5 = 1.6144 \tag{12}$$

$$y_2 = -0.1317 \times 1.2 + 0.9902 \times (-0.8) + (-0.7902) \times 0.5 = -1.5369 \tag{13}$$

$$y_3 = 0.3951 \times 1.2 + (-0.9219) \times (-0.8) + 0.1317 \times 0.5 = 1.5722 \tag{14}$$

Output difference: The errors are small: [0.0245, 0.0639, 0.0375] - about 2.9

5.2 Why Small Errors Don't Matter Much

In neural networks, these outputs typically go through:

- Activation functions (like ReLU, which clips negative values to zero)
- Normalization layers (which adjust the scale)

- Multiple layers of processing (where errors can cancel out)
- Final decision based on relative magnitudes, not absolute values

6. Real-World Impact: The Big Picture

6.1 Memory Savings Analysis

Let's scale our example to real models:

LLaMA-7B Model:

- Parameters: 7 billion
- 32-bit storage: $7 \times 10^9 \times 4$ bytes = 28 GB
- 4-bit storage: $7 \times 10^9 \times 0.5$ bytes = 3.5 GB
- **Reduction: 87.5% less memory needed**

GPT-3 Scale (175B):

- 32-bit: 700 GB (needs expensive server hardware)
- 4-bit: 87.5 GB (fits on high-end consumer GPU)

6.2 Performance Comparison

Aspect	32-bit FP	8-bit INT	4-bit INT
Memory Usage	28 GB	7 GB	3.5 GB
Memory Bandwidth	100%	25%	12.5%
Energy per Operation	1.0x	0.5x	0.3x
Inference Speed	1.0x	1.8x	2.5-4.0x
Model Quality	100%	98-99%	95-97%

Table 1: Comparison across different precisions for LLM inference

6.3 Accuracy Trade-offs in Practice

Real-world benchmark results show quantization is quite practical:

Common Sense Reasoning (HellaSwag):

- 32-bit: 76.1% accuracy
- 4-bit: 73.9% accuracy
- Loss: only 2.2%

Reading Comprehension (MMLU):

- 32-bit: 68.2% accuracy
- 4-bit: 65.8% accuracy
- Loss: only 2.4%

The key insight: humans wouldn't notice the difference in most conversations, but the efficiency gains are enormous.

25% 12.5%
Energy per Operation 1.0x 0.5x 0.3x
Inference Speed 1.0x 1.8x 2.5-4.0x
Model Quality 100% 98-99% 95-97%

Comparison across different precisions for LLM inference

6.3 Accuracy Trade-offs in Practice

Real-world benchmark results show quantization is quite practical:

Common Sense Reasoning (HellaSwag):

- 32-bit: 76.1% accuracy
- 4-bit: 73.9% accuracy
- Loss: only 2.2%

Reading Comprehension (MMLU):

- 32-bit: 68.2% accuracy
- 4-bit: 65.8% accuracy
- Loss: only 2.4%

The key insight: humans wouldn't notice the difference in most conversations, but the efficiency gains are enormous.

7. Advanced Quantization Techniques

7.1 Group-wise Quantization

Instead of using one scale for the entire weight matrix, divide weights into small groups (say, 64 weights each) and quantize each group separately. This is like having multiple thermometers with different scales for different temperature ranges.

Benefits:

- Better captures local weight distributions
- Reduces quantization error by 30-50%
- Minimal additional storage overhead

7.2 Mixed-Precision: Smart Bit Allocation

Not all parts of a neural network are equally sensitive to quantization:

Strategy:

- **Attention layers:** 8-bit (these handle complex relationships)
- **Feed-forward layers:** 4-bit (simpler transformations)
- **Embedding layers:** 16-bit (critical for understanding words)

This is like using different levels of detail in different parts of a map - more detail for complex city centers, less for highways.

7.3 Dynamic vs Static Quantization

- **Static:** Quantization parameters fixed during training
- **Dynamic:** Parameters adjusted based on input data at runtime
- Dynamic is more accurate but computationally expensive

8. Practical Implications and Future Outlook

8.1 Democratizing AI

Quantization has revolutionary implications:

- **Personal devices:** Run 7B parameter models on phones
- **Edge computing:** AI in cars, IoT devices, remote locations
- **Cost reduction:** Lower cloud computing costs
- **Energy efficiency:** Crucial for sustainable AI deployment

8.2 Current Limitations and Research Directions

Challenges:

- Training with quantization is still challenging
- Some tasks (like precise arithmetic) suffer more from quantization
- Hardware support varies across platforms

Cutting-edge Research:

- 2-bit and 1-bit quantization
- Learnable quantization schemes
- Task-adaptive bit allocation
- Quantum-inspired number representations

Key Takeaways

1. Fundamental Trade-off: Quantization offers dramatic efficiency gains (87.5% memory reduction) with minimal quality loss (2-3% accuracy drop).

2. Mathematical Elegance: The linear relationship between bits and precision provides predictable, controllable trade-offs.

3. Practical Revolution: Enables AI deployment from data centers to mobile devices, democratizing access to powerful language models.

4. Robust Foundation: Neural networks' inherent redundancy makes them naturally suited for quantization - unlike traditional software, they gracefully degrade rather than break.

Conclusion

Quantization represents one of the most important practical advances in making AI accessible and sustainable. By understanding how computers represent numbers and applying clever mathematical techniques to reduce precision while preserving intelligence, we've unlocked the ability to run sophisticated language models on everyday devices. This democratization of AI technology promises to bring intelligent systems closer to users while reducing the environmental footprint of artificial intelligence.

The journey from 700 GB models requiring specialized hardware to 3.5 GB models running on consumer devices illustrates how mathematical insights can drive technological revolutions. As we continue to push the boundaries of quantization, we move closer to a future where powerful AI is ubiquitous, efficient, and accessible to all.